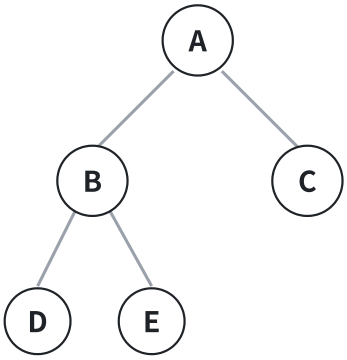
CSED233 · 데이터구조 · 포스텍

정리본 6. Trees Traversal

순회 4가지 · 순회 수열로 트리 재구성 · General Tree 구현 3가지와 이진 트리 변환.

1. Tree Traversal(순회) — 모든 노드를 정확히 한 번씩

- **DFS(깊이 우선)**: 한쪽을 끝까지 먼저. 부모를 언제 방문하느냐에 따라 Preorder / Inorder / Postorder.
- **BFS(너비 우선)**: 같은 깊이부터. Level-order.



이 트리 하나로 아래 4가지 순회를 비교

Preorder(전위) — 부모 → 왼쪽 → 오른쪽
A → B → D → E → C

Inorder(중위) — 왼쪽 → 부모 → 오른쪽
D → B → E → A → C

Postorder(후위) — 왼쪽 → 오른쪽 → 부모
D → E → B → C → A

Level-order(레벨 순회, BFS) — 레벨별로
A → B → C → D → E

DFS 3종은 "부모를 언제 찍느냐"만 다르다

1.1 구현과 용도

```
void preorder(Node* n)  { if (!n) return; cout << n->val; preorder(n->left); preorder(n->right); }
void inorder(Node* n)   { if (!n) return; inorder(n->left); cout << n->val; inorder(n->right); }
void postorder(Node* n) { if (!n) return; postorder(n->left); postorder(n->right); cout << n->val; }
```

순회	자료구조	구현	주요 용도
Preorder	Stack (재귀)	재귀	트리 복사·출력, 파일 시스템 브라우징
Inorder	Stack (재귀)	재귀	**BST에서 오름차순 정렬 출력** (가장 중요한 BST 성질)

Postorder	Stack (재귀)	재귀	트리 삭제(자식 먼저 delete), 후위 표기(postfix)
Level-order	Queue	반복	BFS, 계층별 처리, 최단 경로

```
void levelorder(Node* root) {
  queue<Node*> q;
  if (!root) return;
q.push(root);
  while (!q.empty()) {
    Node* node = q.front(); q.pop();
    cout << node->val;
    if (node->left)  q.push(node->left);
    if (node->right) q.push(node->right);
  }
}
```

Queue(FIFO)라서 같은 깊이의 노드가 먼저 나온다. Stack이면 DFS가 된다.

2. Tree 재구성(Reconstruction)

문제: 순회 수열 하나만으로 원본 트리를 재구성할 수 있는가?

같은 Preorder, 다른 트리

```
Tree 1:   A       Tree 2:   A
        /                   \
       B                     B
      /                       \
     C                         C
```

둘 다 Preorder = A B C → 단일 순회로는 구분 불가능

조합	재구성 가능?
Preorder + Inorder	✓ **가능**
Postorder + Inorder	✓ **가능**
Level-order + Inorder	✓ **가능**

Preorder + Postorder	✗ **불가능** — 왼쪽/오른쪽 서브트리 경계를 모름

→ 일반 이진 트리에서는 Inorder가 반드시 필요하다 (단, Proper 이진 트리는 Preorder + Postorder만으로도 재구성 가능). Inorder가 루트 기준 왼쪽/오른쪽 구간의 경계를 알려주기 때문.

예제 · Preorder + Inorder로 재구성

Pre: `A B D E C` · In: `D B E A C`

① Preorder의 첫 원소는 항상 루트 → 루트 = A.

② Inorder에서 A의 위치를 찾으면 왼쪽 [D B E] / 오른쪽 [C]로 갈라진다.

③ 각 구간을 서브트리로 재귀 재구성: 왼쪽 Pre = B D E, In = D B E → 루트 B, 왼쪽 D, 오른쪽 E.

```
결과:  A
      /\
     B  C
    /\
   D  E
```

3. General Tree(일반 트리) 구현 3가지

3.1 단순 배열 (Parent 포인터 배열)

각 노드의 부모 인덱스만 저장한다.

k	1	2	3	4	5	6
Label	a	b	d	f	c	e
Parent	0	1	2	2	1	2

- Parent(k) 연산은 $O(1)$로 매우 빠름.
- 자식들을 찾으려면 배열 전체 스캔 → $O(n)$. 자식 간 순서(맨 왼쪽 자식)도 표현 못 함.

3.2 List-of-Children

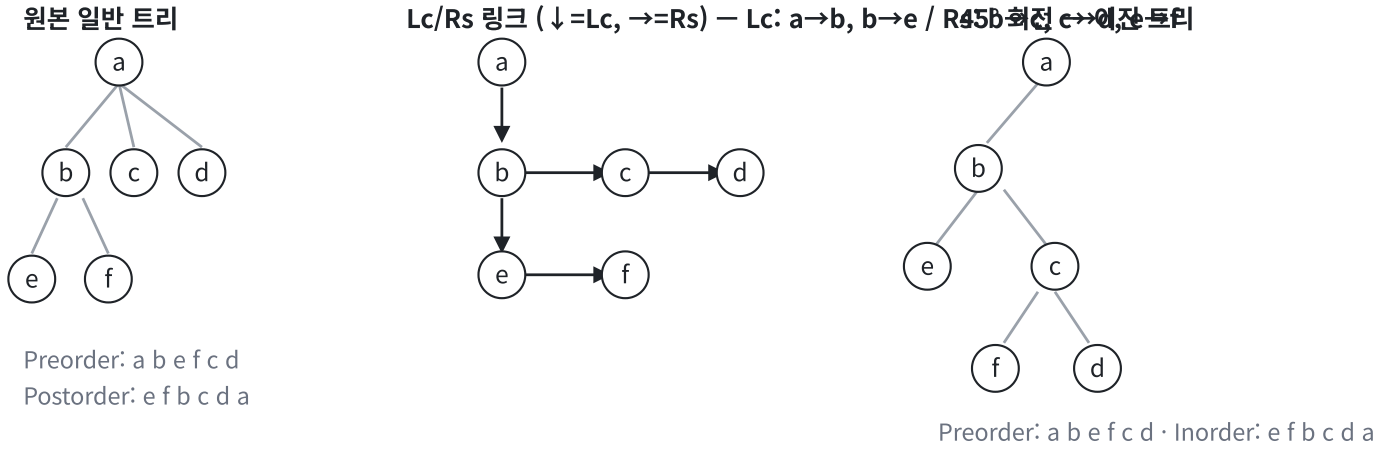
각 부모가 자식 인덱스들의 연결 리스트를 가진다.

k	Label	Parent	Children 리스트
1	a	0	→ 2 → 5
2	b	1	→ 3 → 6 → 4
3	d	2	—
4	f	2	—
5	c	1	—
6	e	2	—

- 두 트리 결합은 새 서브트리 루트의 부모 인덱스만 갱신 (자식 리스트 끝 삽입이 O(1)인 구현에서) → $O(1)$.
- 오른쪽 형제(Right sibling) 접근이 어렵고, 노드 데이터가 배열과 자식 리스트 양쪽에 중복 존재.

3.3 Left-Child / Right-Sibling — 일반 트리를 이진 트리로

노드마다 **맨 왼쪽 자식(Lc)**과 **바로 오른쪽 형제(Rs)**, 딱 2개의 포인터만 쓴다. 자식 수가 아무리 많아도 표현 가능하며, 사실상 일반 트리를 이진 트리로 변환하는 것이다.



Lc = 왼쪽 자식 포인터, Rs = 오른쪽 형제 포인터. 형제 체인을 오른쪽 자식으로 돌리면 이진 트리가 된다.

아름다운 대응 관계

- 일반 트리의 **Preorder** = 변환된 이진 트리의 **Preorder** (a b e f c d — 완벽 일치).
- 일반 트리의 **Postorder** = 변환된 이진 트리의 **Inorder** (e f b c d a — 완벽 일치).

4. Quiz

Q1. BST에서 Inorder의 의미는?

> 힌트

Inorder = 왼쪽(작은 것) → 부모 → 오른쪽(큰 것). BST의 대소 규칙과 맞물려 보자.

> 정답

개념은 힌트 참고.

정답: Inorder 순회 결과가 오름차순 정렬 — BST의 핵심 성질.

Q2. Postorder를 사용하는 대표적인 이유는?

> 힌트

Postorder는 자식이 부모보다 항상 먼저 방문된다. "delete 순서"와 연결해 보자.

> 정답

개념은 힌트 참고.

정답: 트리 삭제 시 자식을 먼저 delete 할 수 있어서 (메모리 누수 방지). 수식 후위 평가에도 사용.

Q3. Preorder + Postorder 조합으로는 재구성이 불가능한 이유는?

> 힌트

루트는 찾을 수 있다. 문제는 왼쪽과 오른쪽의 "경계"다.

> 정답

개념은 힌트 참고.

정답: 루트는 알아도 왼쪽/오른쪽 서브트리의 경계를 구분할 수 없어서. 일반 이진 트리에서는 Inorder가 반드시 필요하다 (Proper 트리라면 Preorder + Postorder로 재구성 가능).

Q4. Level-order 순회에 Queue를 쓰는 이유는?

> **힌트**

FIFO = 먼저 발견한 노드를 먼저 처리. Stack이면 어떻게 될까?

> **정답**

개념은 힌트 참고.

정답: FIFO 덕분에 같은 깊이의 노드를 순서대로 처리할 수 있어서. Stack을 쓰면 DFS가 되어 버린다.